

Project Proposal: Hybrid Access Control System for Cloud-Based Productivity Services

Selected Domain

This project focuses on cloud-based productivity platforms, similar to services provided by Google Workspace. These platforms support communication, collaboration, and document management across organizations, requiring fine-grained and secure access control.

Roles Definition

The system will implement Role-Based Access Control (RBAC) with the following roles:

- **Administrator:** Full control over users, permissions, and services
- **Manager:** Manage team resources, approve access, and share documents
- **Employee:** Create and edit documents, send emails, and collaborate
- **Contractor:** Limited access to assigned documents and tools
- **Viewer:** Read-only access to shared resources

Objects / Resources Definition

The system will control access to key productivity services:

- Google Docs (documents and reports)
- Gmail (organizational communication)
- Google Drive (file storage and sharing)
- Google Calendar (meetings and scheduling)

Access Rights

Permissions will include:

- **Read:** View documents, emails, and events
- **Write:** Create or modify documents and emails
- **Delete:** Remove files or messages
- **Share:** Grant access to others
- **Send:** Send emails or calendar invites
- **Admin:** Manage users, roles, and system settings

Attributes (ABAC Integration)

To enhance flexibility, Attribute-Based Access Control (ABAC) will complement RBAC using:

- **User attributes:** department, role level, employment type (full-time vs contractor)
- **Resource attributes:** document owner, sensitivity level (public/internal/confidential)
- **Context attributes:** time of access, device type, login location

Policy Architecture (PAP, PDP, PEP)

The system follows a standard policy-based access control architecture:

- **Policy Administration Point (PAP):**
Managed by Administrators to define roles, permissions, and attribute-based policies (e.g., who can access confidential documents).
- **Policy Decision Point (PDP):**
Core engine that evaluates access requests by combining RBAC and ABAC rules and returns a permit/deny decision.
- **Policy Enforcement Point (PEP):**
Integrated into the application layer (e.g., document access or email sending). It intercepts user requests, queries the PDP, and enforces the decision.

Example Policy Rules

- Employees can edit Google Docs they own or are explicitly assigned
- Contractors can only read or comment on documents shared with them
- Managers can share Google Drive files externally, but Employees cannot
- Access to confidential documents requires matching department + high clearance
- Users can only send Gmail messages from organizational accounts
- No user can both grant access permissions and audit access logs (Separation of Duties)

Implementation Plan

The system will be implemented as a Python-based web application:

- **Backend:** Python (Flask or FastAPI) for access control logic
- **Data Storage:** SQLite or JSON for users, roles, permissions, and attributes
- **Core Components:**
 - Role definitions and user-role assignments
 - Permission mapping for each resource
 - Attribute-based policy evaluation engine
 - Access decision function combining RBAC + ABAC

Access Decision Function

Each access request is evaluated as follows:

1. PEP intercepts request
2. PDP evaluates RBAC permissions
3. PDP evaluates ABAC attributes
4. Decision returned to PEP → access granted or denied